

Lecture 16:

Introduction to Deep Learning

Gradient Descent

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- Recall: Loss Functions and Activation Functions
- Gradient Descent
- Back Propagation

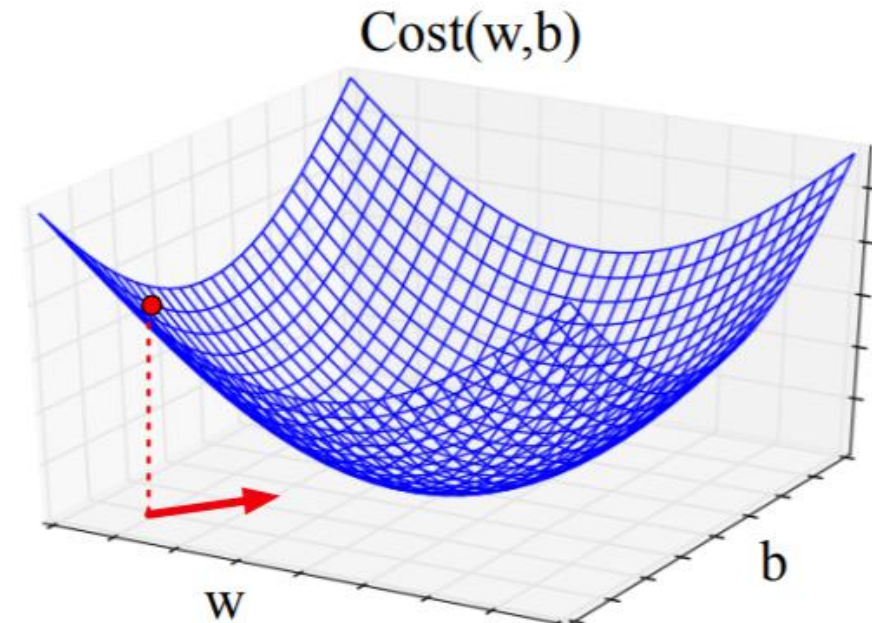
Gradient Descent

- n dimensional

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^m L_{\text{CE}}(f(x^{(i)}; \theta), y^{(i)})$$

$$\theta_{t+1} = \theta_t - \eta \nabla L(f(x; \theta), y)$$

$$\nabla_{\theta} L(f(x; \theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x; \theta), y) \\ \frac{\partial}{\partial w_2} L(f(x; \theta), y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x; \theta), y) \\ \frac{\partial}{\partial b} L(f(x; \theta), y) \end{bmatrix}$$



Standard Gradient Descent

```
while true,  
     $\theta_t = \theta_{t-1} - \eta_t \Delta(\theta_{t-1})$   
    if  $||\theta_t - \theta_{t-1}|| < \epsilon$  then break;
```

Batch Gradient Descent

- Use the *entire dataset* to compute the gradient at every iteration.

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta_t) \quad \text{where} \quad J(\theta) = \frac{1}{N} \sum_{i=1}^N L(f(x_i; \theta), y_i)$$

- Think of this as computing the *exact slope* of the loss function before every update.
- Since we use all samples, the update direction is perfectly accurate, but slow to compute.
- **Example:**
If your dataset has 1 million samples, you must process *all of them* to make just one update.

Batch Gradient Descent

- **Pros:**

- Smooth and stable convergence
- Works well on small datasets

- **Cons:**

- Extremely slow for large datasets
- Requires a lot of memory

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} J(\theta_t)$$

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N L(f(x_i; \theta), y_i)$$

Stochastic Gradient Descent (SGD)

- Use *one random sample* at a time to estimate the gradient.

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L(f(x_i; \theta), y_i)$$

- Instead of waiting to see the whole dataset, we take an immediate small step after each sample.
- This introduces randomness: the path “zigzags,” but moves roughly downhill on average.
- Noisy updates, helps escape shallow local minima.
- Requires many iterations , but each iteration is very cheap.

Stochastic Gradient Descent (SGD)

- **Pros:**

- Very fast updates
- Can handle streaming data
- Helps generalization via noise

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L(f(x_i; \theta), y_i)$$

- **Cons:**

- Very noisy convergence
- Needs good learning rate tuning

Mini-Batch Gradient Descent

- Compute gradient on a *small random subset* of data (size = batch size, e.g., 32-256).

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{m} \sum_{j=1}^m \nabla_{\theta} L(f(x_j; \theta), y_j)$$

- Combines benefits of batch (stability) and stochastic (speed).
- Exploits parallelism: GPU computes gradients efficiently on mini-batches.
- Reduces noise but keeps updates frequent.
- **Typical batch sizes:** 32, 64, 128 (if too large, it behaves similar to full batch GD).

Mini-Batch Gradient Descent

- **Pros:**

- Most efficient in practice
- Smooth convergence
- Scales well on GPUs

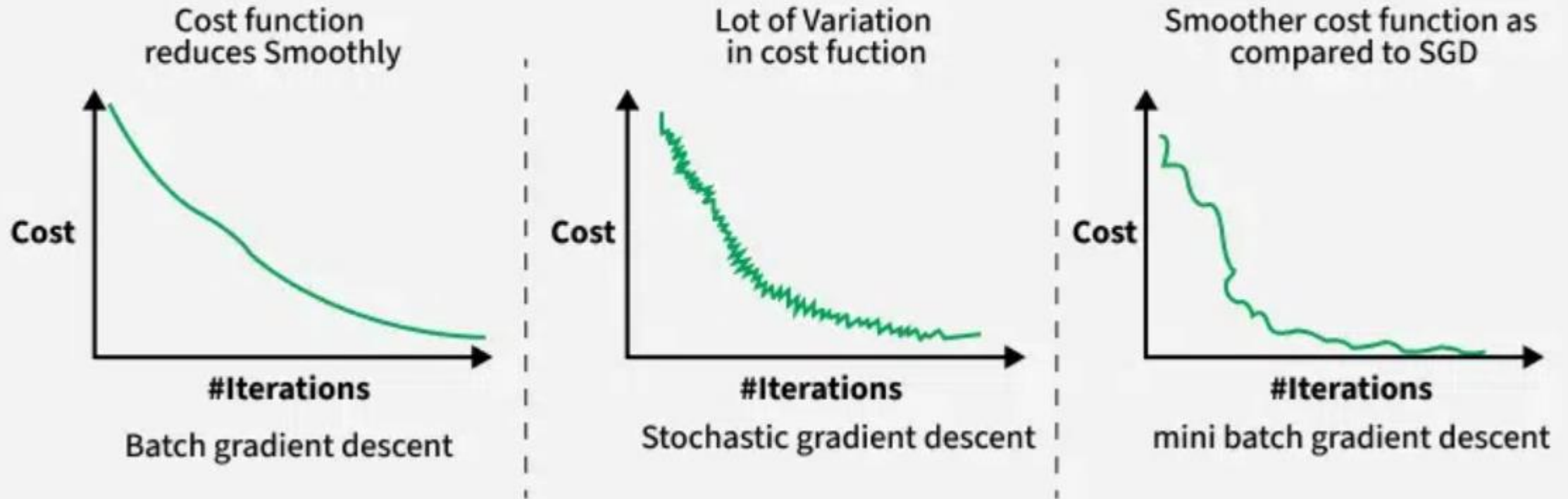
$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{m} \sum_{j=1}^m \nabla_{\theta} L(f(x_j; \theta), y_j)$$

- **Cons:**

- Need to tune batch size and learning rate

Comparison

Understanding Batch Size in Neural Network



Different Optimization Techniques

- **Gradient Descent:** the most basic but the most used optimization algorithm to find parameters (e.g., weights and biases).
- It is a first-order optimization algorithm, dependent on the first order derivative of a loss function.

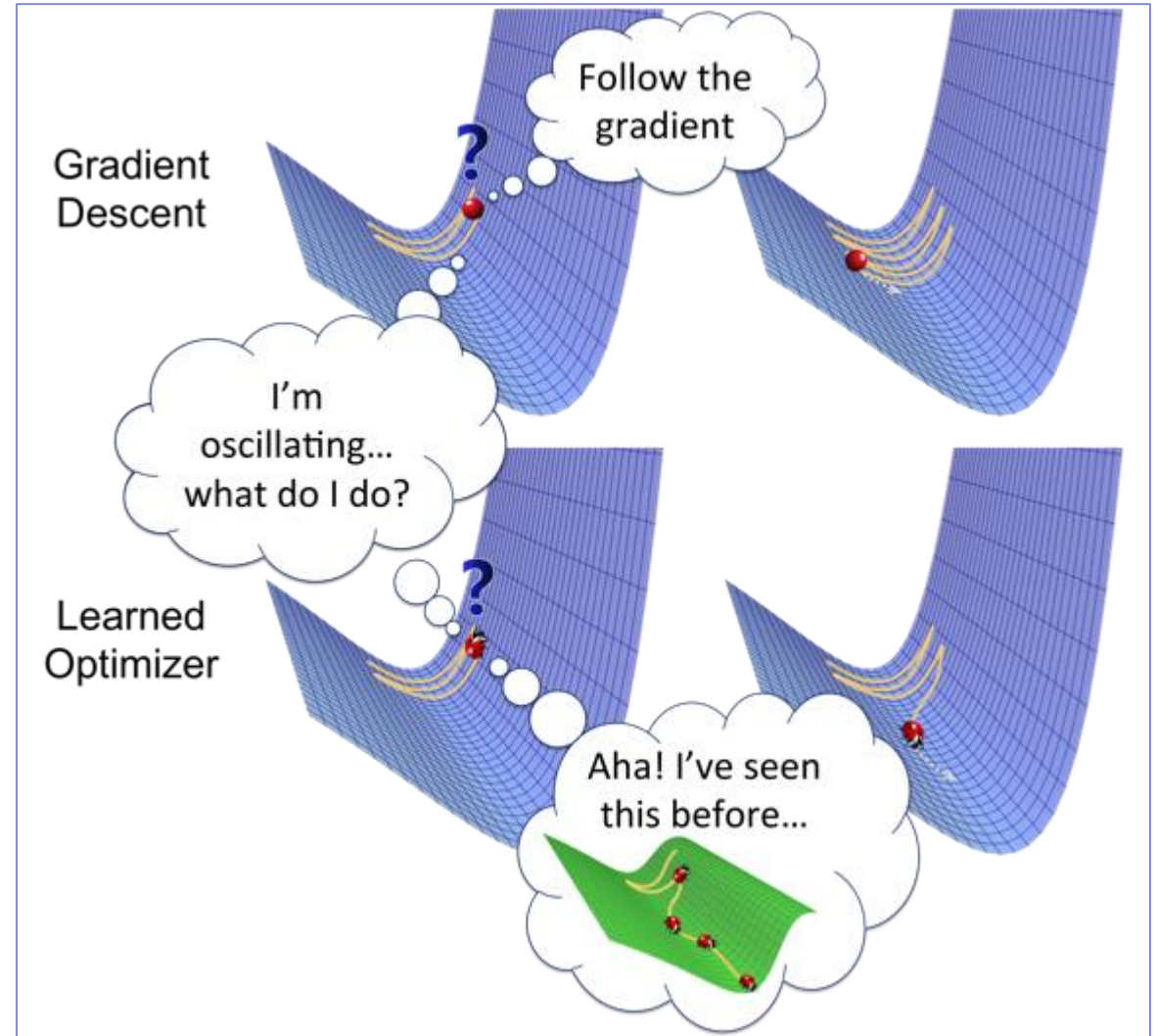
$$\theta = \theta - \alpha \cdot \nabla J(\theta)$$

- **Pros:** Easy to computation, implement, and understand.
- **Cons:** May trap at local minima, long time to converge to the minima, needs large memory.

Challenges of all versions of gradient descent

- Selecting an optimum value of the learning rate.
- They have a constant learning rate for all the parameters (not all parameters should be changes at the same rate).
- May trap at local minima.
- **How to address challenges of gradient descent?**

Use Momentum



Gradient descent optimization algorithms

- We have many different versions of gradient descent optimization algorithms:
 - Momentum
 - Nesterov accelerated gradient
 - Adagrad
 - Adadelta
 - RMSprop
 - **Adam**
 - AdaMax
 - Nadam
 - AMSGrad

Momentum-based Gradient Descent

- Use past gradients to **accelerate** learning: similar to giving the gradient some “inertia”.

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

- v_t : **velocity** (exponentially decaying average of past gradients)
 - β : momentum coefficient (commonly 0.9)
- Prevents oscillations along directions with steep curvature.
- Accelerates movement along shallow directions.
- Think of a heavy ball rolling down a hill: once it gains speed, it is less affected by small bumps.

Momentum-based Gradient Descent

- **Pros:**

- Faster convergence
- Helps overcome local minima
- Smoother trajectory

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

- **Cons:**

- Can overshoot if η or β too large

Momentum term adds “memory”: smoother and faster updates.

Nesterov Accelerated Gradient (NAG)

Improves Momentum by looking **ahead** before computing gradient.

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t - \beta v_{t-1})$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

- First, estimate where parameters will be after momentum step.
- Then compute gradient at that “lookahead” position, therefore, more accurate correction.

Nesterov Accelerated Gradient (NAG)

- **Pros:**

- Anticipates direction, therefore, reduces overshoot
- Converges faster than plain Momentum

- **Cons:**

- Slightly more computation per iteration

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla_{\theta} J(\theta_t - \beta v_{t-1})$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

Instead of blindly rolling downhill, NAG looks a few steps ahead to see if it is still downhill.

AdaGrad (Adaptive Gradient)

Each parameter gets its **own adaptive learning rate**, based on its historical gradients.

$$r_t = r_{t-1} + (\nabla_{\theta} J(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t + \epsilon}} \nabla_{\theta} J(\theta_t)$$

where:

- r_t : running sum of squared gradients (per parameter)
- ϵ : small constant to avoid divide-by-zero
- Parameters with large gradients: results in smaller learning rate
- Parameters with small gradients: results in larger learning rate

AdaGrad (Adaptive Gradient)

- **Pros:**

- Ideal for sparse features (e.g., NLP)
- Automatically tunes step sizes

- **Cons:**

- Accumulated r_t grows indefinitely.....learning rate keeps shrinking
.....training can stop early

$$r_t = r_{t-1} + (\nabla_{\theta} J(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t + \epsilon}} \nabla_{\theta} J(\theta_t)$$

AdaGrad learns aggressively at first, but slows down too much later.

RMSProp

- Fixes AdaGrad's decaying learning rate problem using an **exponential moving average**.

$$r_t = \beta r_{t-1} + (1 - \beta)(\nabla_{\theta} J(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t + \epsilon}} \nabla_{\theta} J(\theta_t)$$

- r_t : exponentially weighted moving average of squared gradients
 - Typical $\beta = 0.9$
- Keeps track of *recent* gradient magnitudes, not the entire history.
- Allows learning rate to stay adaptive but not shrink forever.

RMSProp

- **Pros:**
 - Works well in non-stationary settings
 - Very effective in RNNs and deep nets
- **Cons:**
 - Hyperparameter tuning still important

$$r_t = \beta r_{t-1} + (1 - \beta)(\nabla_{\theta} J(\theta_t))^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t + \epsilon}} \nabla_{\theta} J(\theta_t)$$

RMSProp is AdaGrad with short-term memory: adapts but does not forget.

Adam (Adaptive Moment Estimation)

- Combines **Momentum + RMSProp**: maintains both *mean* and *variance* of gradients.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} J(\theta_t)$$

$$r_t = \beta_2 r_{t-1} + (1 - \beta_2) (\nabla_{\theta} J(\theta_t))^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{r}_t = \frac{r_t}{1 - \beta_2^t}$$

- Typical values: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$
- m_t : smooths direction (momentum)
- r_t : scales by variance (adaptive rate)
- Bias correction (hats) ensures stability early in training

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{r}_t} + \epsilon}$$

Adam (Adaptive Moment Estimation)

- **Pros:**

- Works well out of the box
- Adaptive + momentum = robust
- Default in PyTorch/TensorFlow

- **Cons:**

- Can sometimes overfit or fail to generalize
- Slightly more complex math

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} J(\theta_t)$$

$$r_t = \beta_2 r_{t-1} + (1 - \beta_2) (\nabla_{\theta} J(\theta_t))^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{r}_t = \frac{r_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{r}_t} + \epsilon}$$

Adam is the all-in-one optimizer. Most deep learning models use it unless there is a reason not to.

From Gradient Descent to Newton's Method

- Gradient Descent update:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

- uses **first-order information** (slope).
- Idea: use curvature (second derivative) to take *smarter* steps.
- Newton's Method (1 D):

$$\theta_{t+1} = \theta_t - \frac{J'(\theta_t)}{J''(\theta_t)}$$

Gradient descent tells us which way to go.
Newton's method also asks *how curved* the terrain is. if it is steep, take a small step; if it is flat, take a big one.

Newton's Method in Many Dimensions

- Multivariate objective $J(\theta)$
 - Gradient: $\nabla_{\theta}J(\theta)$
 - Hessian: $H(\theta) = \nabla_{\theta}^2J(\theta)$
- Update rule:

$$\theta_{t+1} = \theta_t - H(\theta_t)^{-1} \nabla_{\theta} J(\theta_t)$$

- H^{-1} rescales each direction by local curvature.

Newton's step adjusts both *direction* and *magnitude* of movement. On narrow valleys, it automatically shortens steps along steep axes and lengthens them along flat ones.

Newton's Method in Many Dimensions

- Multivariate objective $J(\theta)$

- Gradient: $\nabla_{\theta} J(\theta)$

- Hessian: $H(\theta) = \nabla_{\theta}^2 J(\theta)$

- Update rule:

$$\theta_{t+1} = \theta_t - H(\theta_t)^{-1} \nabla_{\theta} J(\theta_t)$$

- H^{-1} rescales each direction by local curvature.

Newton's step adjusts both *direction* and *magnitude* of movement.

On narrow valleys, it automatically shortens steps along steep axes and lengthens them along flat ones.

If Newton's method converges faster, why don't we use it in deep learning?

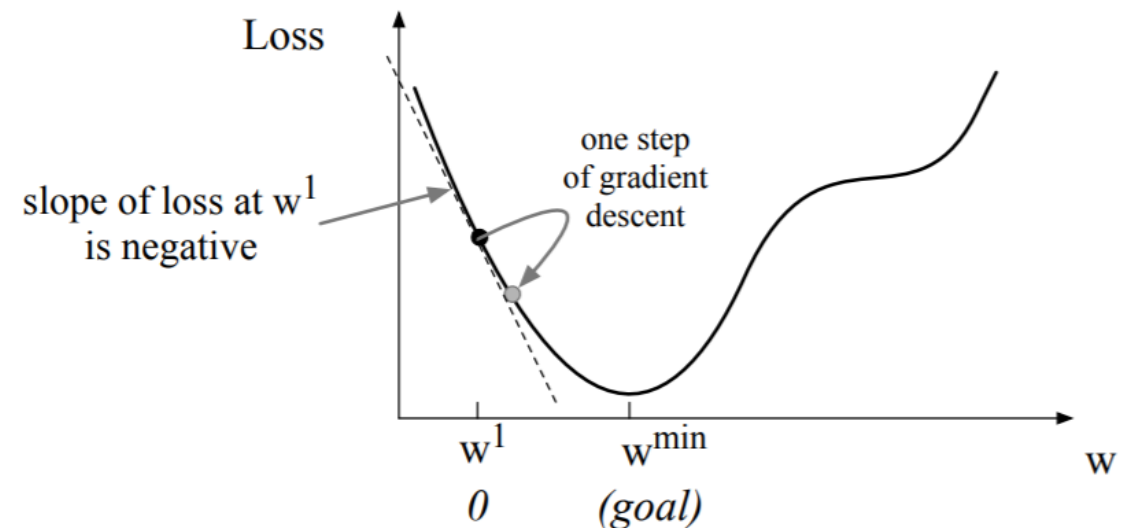
Why Not using Newton's Method?

- Mathematically, Newton's method is beautiful. It knows both the slope and the shape of the loss. Near a minimum, it can converge in just a handful of steps.
- **The Problem: The Hessian Is Enormous**
 - Deep nets have **millions of parameters** Hessian is $n \times n$.
 - Example: 10^6 parameters, meaning 10^{12} Hessian entries.
 - Computing and storing H : **impossible in practice**.
 - Inverting H : $O(n^3)$ operations: astronomically expensive.
- For a modern network, the Hessian would require terabytes of memory.

Review of Gradient Descent

- In the case of logistic regression $\theta = w, b$
- The gradient descent algorithm finds the gradient of the loss function at the current point and moves in the opposite direction.
- The magnitude of move is the value of the slope $\frac{d}{dw}L(f(x; w), y)$ weighted by a learning rate η

$$w^{t+1} = w^t - \eta \frac{d}{dw}L(f(x; w), y)$$



How to calculate gradient in gradient descent?

- **Backpropagation algorithm:** The term backpropagation refers to the algorithm for computing the gradient.
- The **gradient** is used in the gradient descent algorithm.

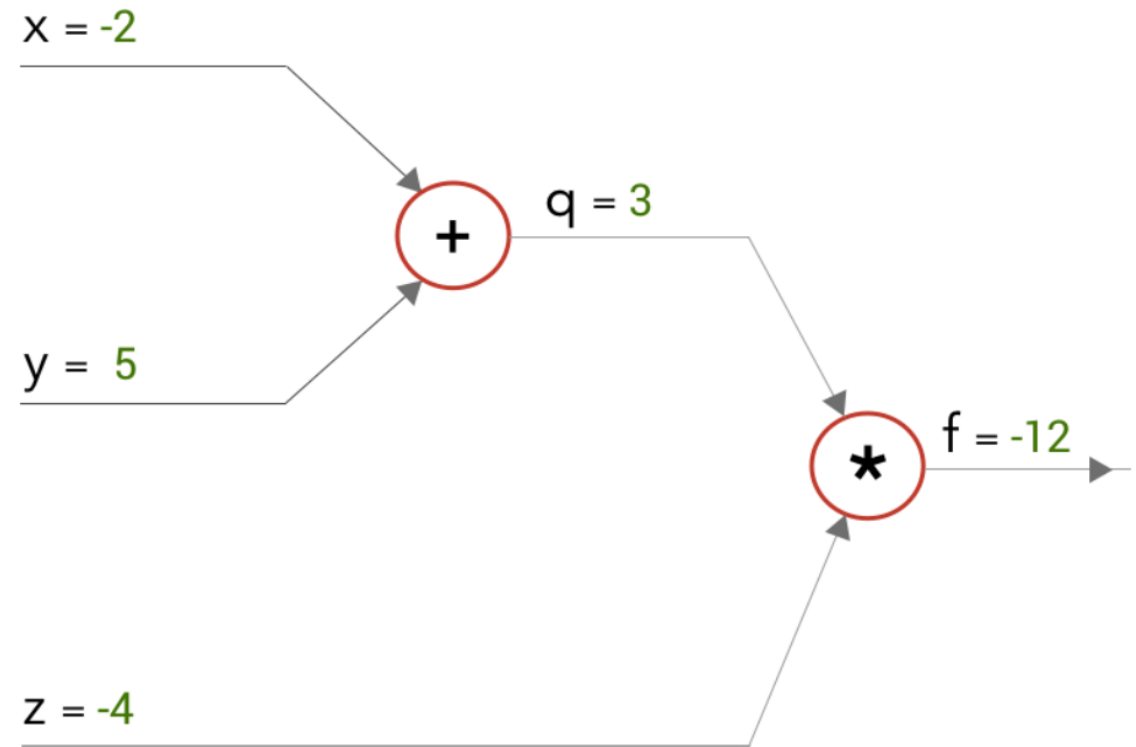
Backpropagation

- Understanding Chain Rule
- **Example:**

$$f(x, y, z) = (x + y)z$$

$$q = x + y$$

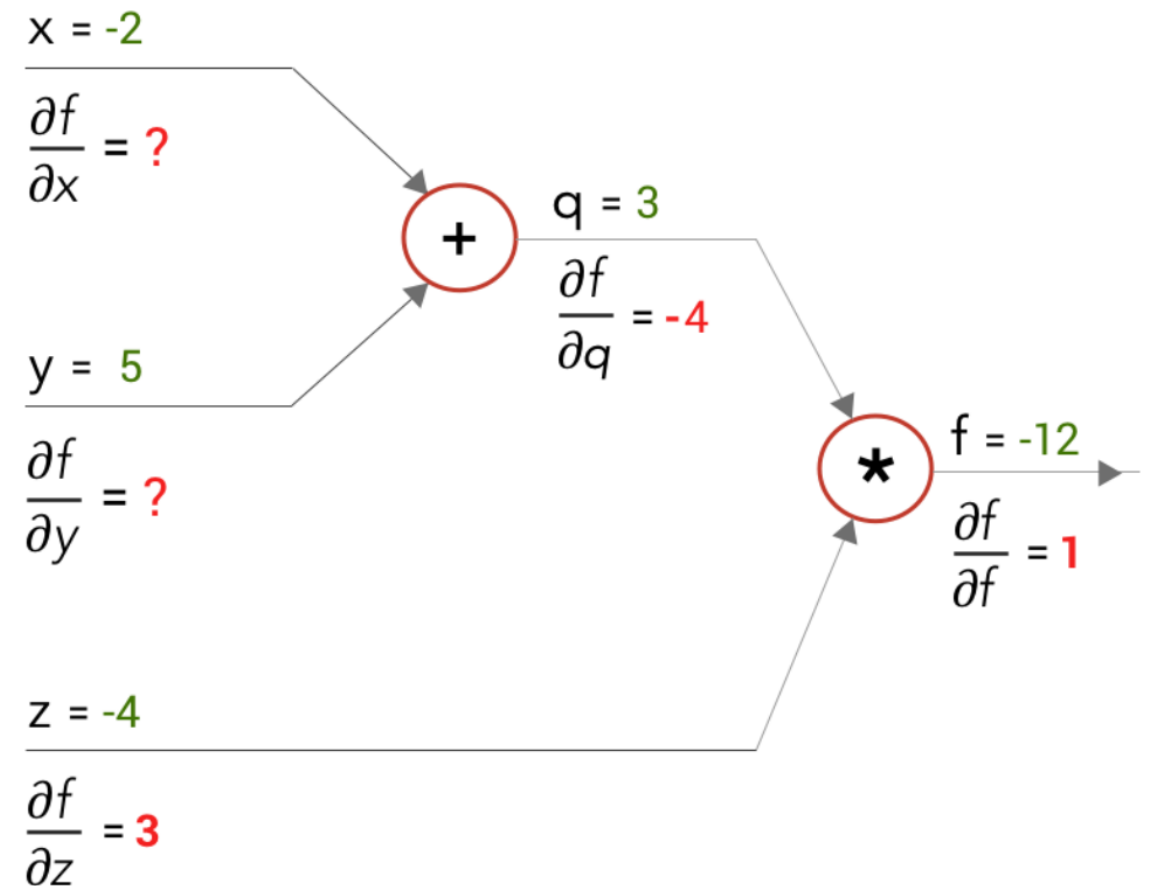
$$f = q * z$$



Backpropagation

- How to calculate $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$ and $\frac{\partial f}{\partial z}$?

$f = q * z$ $\frac{\partial f}{\partial q} = z \mid z = -4$ $\frac{\partial f}{\partial z} = q \mid q = 3$	$q = x + y$ $\frac{\partial q}{\partial x} = 1 \quad \frac{\partial q}{\partial y} = 1$
--	--



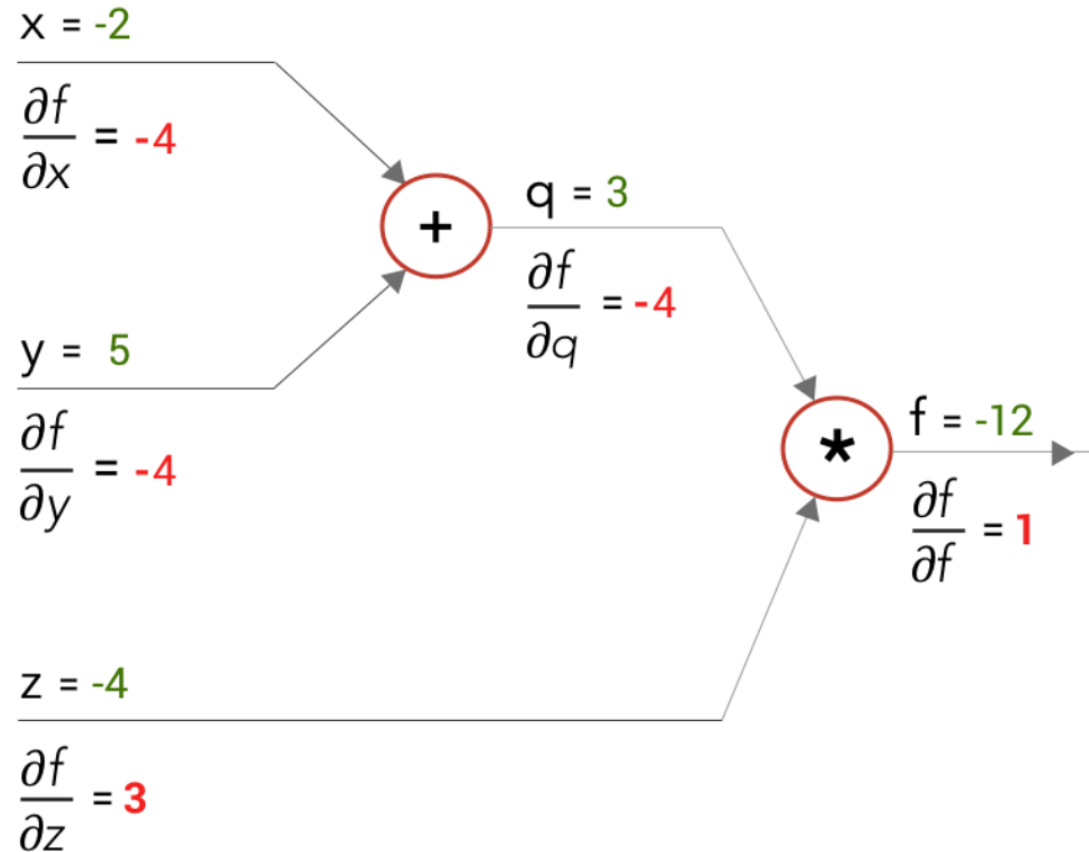
Backpropagation

- Using the Chain Rule

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} * \frac{\partial q}{\partial x}$$

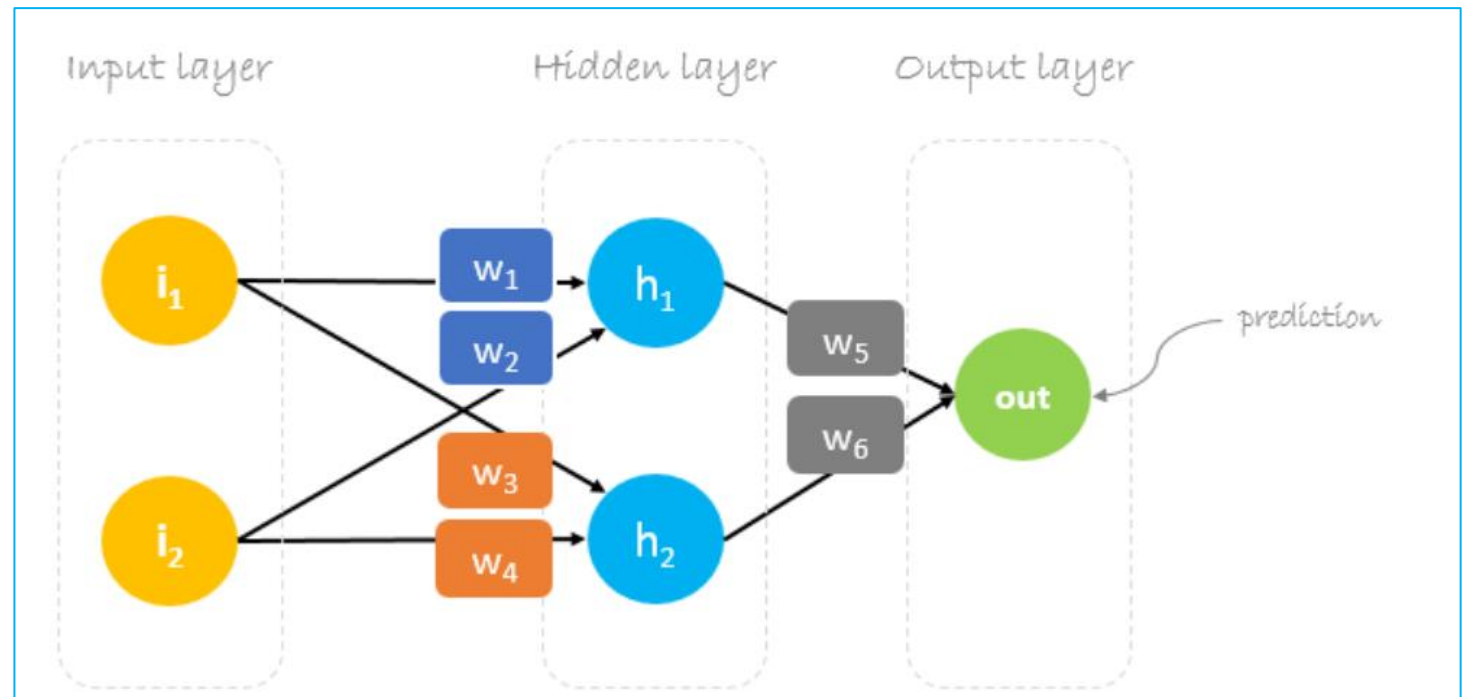
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} * \frac{\partial q}{\partial x} = -4 * 1 = -4$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} * \frac{\partial q}{\partial y} = -4 * 1 = -4$$



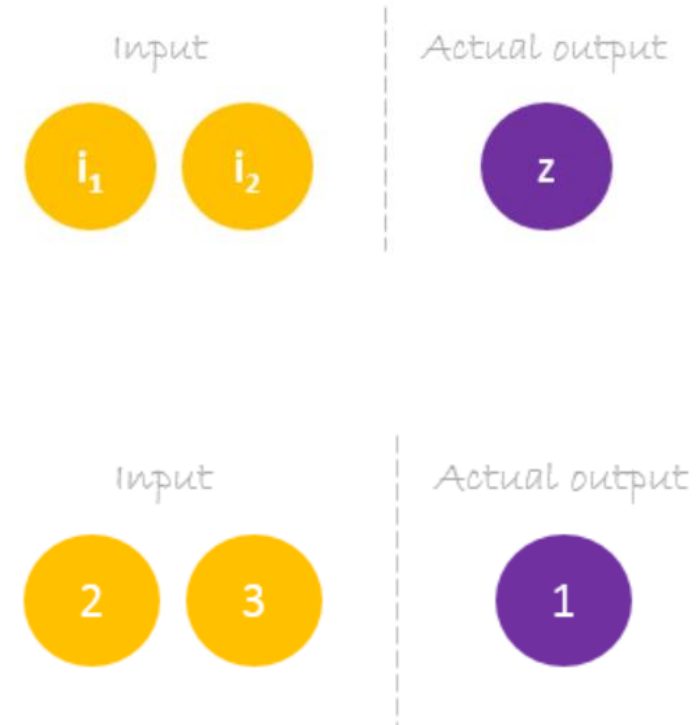
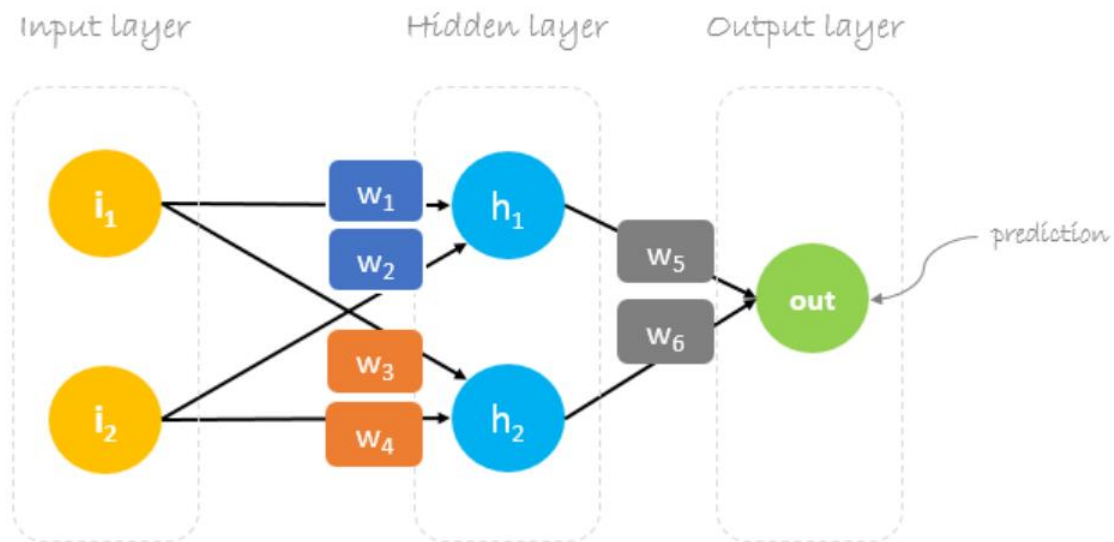
Example

- The initial weights: $w_1 = 0.11$, $w_2 = 0.21$, $w_3 = 0.12$, $w_4 = 0.08$, $w_5 = 0.14$ and $w_6 = 0.15$

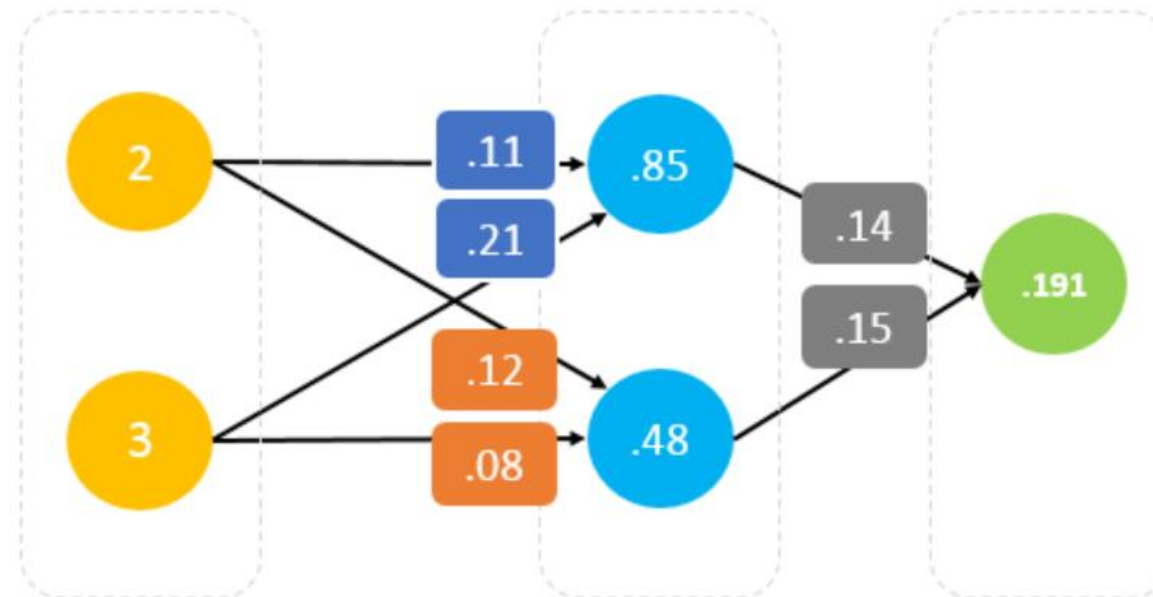


Dataset

- inputs=[2, 3] and output=[1].



Forward Pass



Forward Pass

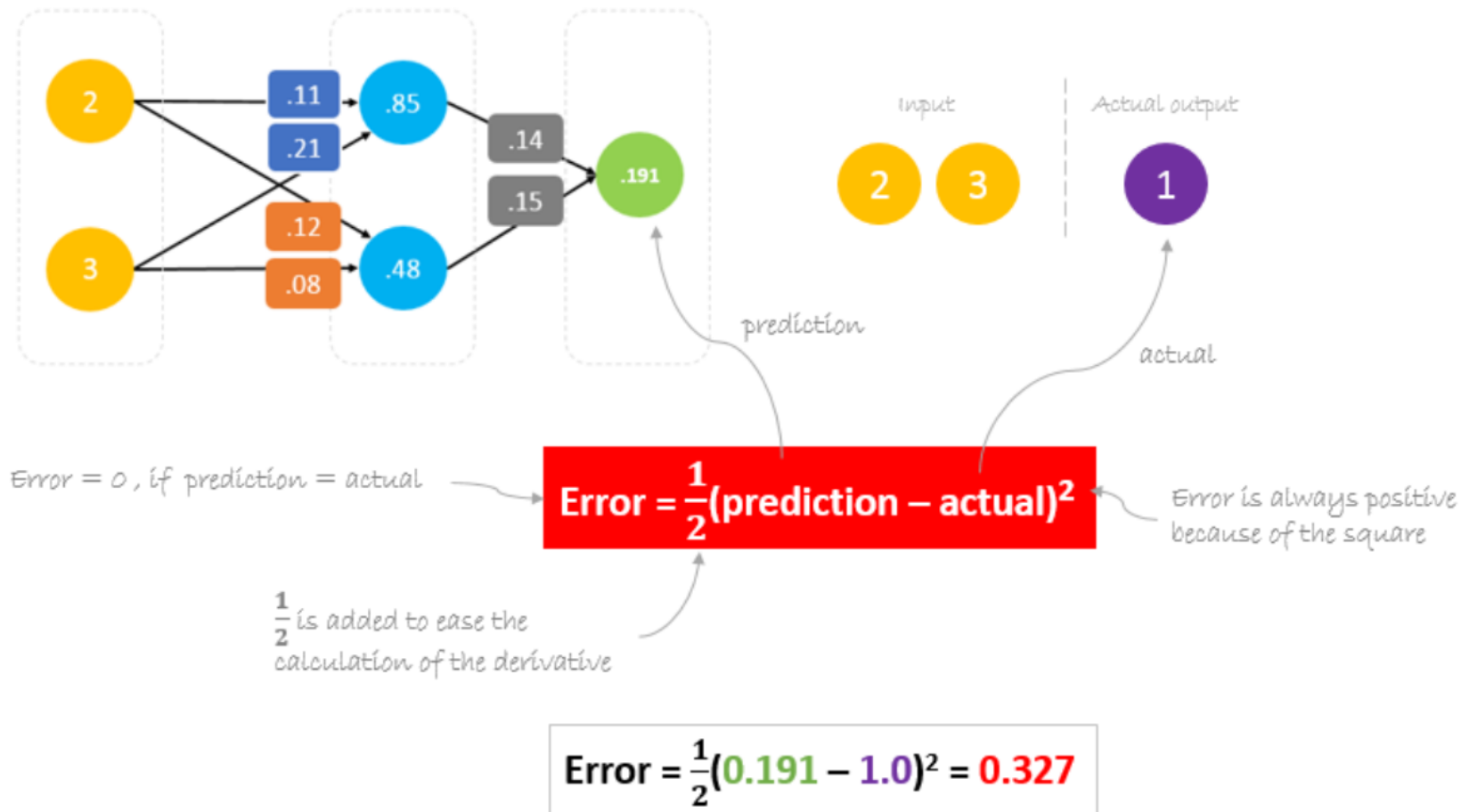
$$\begin{bmatrix} 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} = \begin{bmatrix} 0.85 & 0.48 \end{bmatrix} \cdot \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = \begin{bmatrix} 0.191 \end{bmatrix}$$

Matrix multiplication

Details

$$2 \times .11 + 3 \times .21 = .85$$
$$2 \times .12 + 3 \times .08 = .48$$
$$.85 \times .14 + .48 \times .15 = .191$$

Calculating Error



Reducing Error

$$\text{prediction} = \text{out}$$



$$\text{prediction} = (h_1) w_5 + (h_2) w_6$$



$$\begin{aligned} h_1 &= i_1 w_1 + i_2 w_2 \\ h_2 &= i_1 w_3 + i_2 w_4 \end{aligned}$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

Backpropagation

$$^*W_x = W_x - a \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

Diagram illustrating the weight update formula during backpropagation:

- *W_x : New weight
- W_x : Old weight
- a : Learning rate
- $\left(\frac{\partial \text{Error}}{\partial W_x} \right)$: Derivative of Error with respect to weight

Backpropagation

$$*W_6 = W_6 - \mathbf{a} \left(\frac{\partial \text{Error}}{\partial W_6} \right)$$

$$\frac{\partial \text{Error}}{\partial W_6} = \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial W_6} \quad \leftarrow \text{chain rule}$$

$$\text{Error} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

$$\frac{\partial \text{Error}}{\partial W_6} = \frac{1}{2}(\text{prediction} - \text{actual})^2 * \frac{\partial (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial W_6}$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

$$\frac{\partial \text{Error}}{\partial W_6} = 2 * \frac{1}{2}(\text{prediction} - \text{actual}) \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} * (i_1 w_3 + i_2 w_4) \quad \leftarrow h_2 = i_1 w_3 + i_2 w_4$$

$$\frac{\partial \text{Error}}{\partial W_6} = (\text{prediction} - \text{actual}) * (h_2)$$

$$\Delta = \text{prediction} - \text{actual} \quad \leftarrow \text{delta}$$

$$\frac{\partial \text{Error}}{\partial W_6} = \Delta h_2$$

Backpropagation

$$*W_6 = W_6 - \mathbf{a} \left(\frac{\partial \text{Error}}{\partial W_6} \right)$$

$$*W_6 = W_6 - \mathbf{a} \Delta h_2$$

$$*W_5 = W_5 - \mathbf{a} \Delta h_1$$

Backpropagation

- Similar procedure for w_1 , w_2 , w_3 and w_4

$$\frac{\partial \text{Error}}{\partial w_1} = \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial h_1} * \frac{\partial h_1}{\partial w_1} \quad \leftarrow \text{chain rule}$$

$$\frac{\partial \text{Error}}{\partial w_1} = \frac{\partial \frac{1}{2}(\text{prediction} - \text{actual})^2}{\partial \text{prediction}} * \frac{\partial (h_1) w_5 + (h_2) w_6}{\partial h_1} * \frac{\partial i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial \text{Error}}{\partial w_1} = 2 * \frac{1}{2} (\text{prediction} - \text{actual}) \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} * (w_5) * (i_1)$$

$$\frac{\partial \text{Error}}{\partial w_1} = (\text{prediction} - \text{actual}) * (w_5 i_1)$$

$$\frac{\partial \text{Error}}{\partial w_1} = \Delta w_5 i_1$$

$$\text{Error} = \frac{1}{2} (\text{prediction} - \text{actual})^2$$

$$\text{prediction} = (h_1) w_5 + (h_2) w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

$$\Delta = \text{prediction} - \text{actual} \quad \leftarrow \text{delta}$$

Backpropagation

updated weights

$$\begin{aligned}
 *w_6 &= w_6 - a (h_2 \cdot \Delta) \\
 *w_5 &= w_5 - a (h_1 \cdot \Delta) \\
 *w_4 &= w_4 - a (i_2 \cdot \Delta w_6) \\
 *w_3 &= w_3 - a (i_1 \cdot \Delta w_6) \\
 *w_2 &= w_2 - a (i_2 \cdot \Delta w_5) \\
 *w_1 &= w_1 - a (i_1 \cdot \Delta w_5)
 \end{aligned}$$

- We can rewrite in the form of matrices:

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a \Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a \Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} \cdot \begin{bmatrix} w_5 & w_6 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a i_1 \Delta w_5 & a i_1 \Delta w_6 \\ a i_2 \Delta w_5 & a i_2 \Delta w_6 \end{bmatrix}$$

Finding New Weights: Backward Pass

Finding the new weights using derived formulas

$$\Delta = 0.191 - 1 = -0.809 \quad \leftarrow \text{Delta} = \text{prediction} - \text{actual}$$

$$a = 0.05 \quad \leftarrow \text{Learning rate, we smartly guess this number}$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} \cdot [0.14 \quad 0.15] = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$

Forward Pass

- using the new weights, repeat the forward pass.
- **Outcome:** The prediction 0.26 is a little closer to actual output than previous 0.191.
- **Next:** repeat the same process of backward and forward pass until error is close or equal to zero.

